Complemento da Auditoria do Portfólio GitHub

Edição revisada após as correções locais. Este complemento conserva o diagnóstico da segunda rodada; não descreve automaticamente o código já corrigido. O relatório principal reúne os 47 IDs na Parte 5. A revisão por repositório e imagem distingue o que foi confirmado, corrigido novamente e não reexecutado. As versões anteriores estão preservadas em `_revisao_conformidade_2026-10-13/antes-*.md`. Não houve publicação destas alterações no GitHub.

Perfil: arthurjoanes
Data: 13 de outubro de 2026

Esta é a leitura separada da Parte 8 do relatório principal.

Parte 8 - Revisão complementar: segurança, operação e contratos

Rodada 2: 13 de outubro de 2026. Complemento solicitado após a primeira entrega. O inventário público foi consultado novamente e mantém os mesmos 11 repositórios. O relatório inicial foi preservado em `_auditoria_2026-10-13/baseline/`. Os IDs anteriores permanecem válidos.

Esta rodada acrescentou **9 achados**, sendo 0 P0, 3 P1 e 6 P2, chegando a **47 achados** (24 P1 e 23 P2). Na edição inicial, o plano estava dividido entre 38 itens e este complemento. Na edição revisada, a Parte 5 do relatório principal reúne os 47 IDs; esta Parte 8 preserva o diagnóstico, o recorte e a checklist da segunda rodada.

Objetivo e limites desta rodada

A revisão voltou aos projetos por outras frentes: configuração e segredos, concorrência e orçamento de recursos, autorização e isolamento, integridade de arquivos, privacidade e recuperação, observabilidade, migrações e entrega contínua. Os achados anteriores de UI e documentação não foram contados outra vez. O número de problemas novos não mede a profundidade de um repositório: quando uma hipótese já estava tratada, foi registrada como controle observado.

O Docker Linux continua indisponível. Provas novas usam funções reais com substituições delimitadas, arquivos temporários ou adaptação de dados, sem efetuar restauração real, provocar OOM, encher disco, chamar LLM pago ou gerar carga contra sistemas externos. Não há nova validação visual nesta rodada. Os estados do GitHub foram consultados em leitura; os repositórios originais não foram editados.

As notas da primeira rodada são avaliações por dimensão, não contagens de defeitos; não foram recalculadas mecanicamente. O novo problema de confiança em backup e os limites de concorrência entram na prioridade operacional junto dos P1 anteriores. Não foi feita varredura exaustiva de CVEs de todos os lockfiles/imagens nem certificação de privacidade.

Cobertura complementar por repositório

Repositório	Tópicos adicionais	Resultado
arthurjoanes	Proveniência e comunicação pública	Nenhum novo defeito confirmado nos caminhos revisitados
evidencedesk	Autorização e isolamento por tenant, Sessão, Origin e CSRF, Privacidade de erros, logs e telemetria, Limites de upload e interpretação, Retenção, limpeza e recuperação, Reserva de custo e liberação de modelos, CI, dependências e configuração	EVD-07
gestao-recebiveis	Transações, concorrência e idempotência financeira, Sessão, CSRF, papéis e admissão de login, Importação integral e validação de datas/dinheiro, Lease, fencing, envio incerto e conciliação de lembretes, Privilégios PostgreSQL e migração financeira, Backup isolado, privacidade de evidências e guards de limpeza, CI, containers, locks e contratos de adaptação de dados	REC-07, REC-08
loja-assistente	Autorização por tenant e loja, Sessão, Origin, CSRF e proxy, Privacidade de logs e erros de configuração, Orçamento de recursos de autenticação, Limites de corpo e de uso de LLM, Retenção e limpeza, CI, configuração e dependências	LOJ-07, LOJ-08
varejo-data-pipeline	Consistência entre versões Delta e ponto de publicação, Idempotência, revisões e conflitos de uma venda, Validação de dinheiro, timestamps, manifests e cópia física de entrada, Backup integral, verificação de hashes e retenção conservadora, Proveniência OpenLineage e reconstrução a partir de commits duráveis, CI, isolamento de containers e dependências JVM fixadas	Nenhum novo defeito confirmado nos caminhos revisitados
api-sentinel	Autenticação, autorização por organização/loja/escopo e identidade do cursor, Segredos, sanitização de logs e exceções, Destino ERP/probe e referências remotas da ferramenta de contratos, Limites de entrada, recepção do corpo e cancelamento, Migração inicial, emissão/revogação e papéis de banco, Docker/Compose, CI, dependências, scan e escopo de publicação	Nenhum novo defeito confirmado nos caminhos revisitados
containerops	Autorização por proprietário, limites HTTP e sanitização de logs, Segredos de operação, Compose e exposição de rede, Migrações expansivas, backup, restore e rollback, Integridade de pacote portátil, confiança na origem e recuperação de importação, Catálogo, retenção e journal, Arquivo de imagem OCI/Docker, supply chain, CI, scan e dependências	OPS-05
cycleops	Configuração e limites de recursos, Retenção de dados, Segurança de rede e logs, CI/CD	CYC-04, CYC-05

Repositório	Tópicos adicionais	Resultado
quake-exposure	Integridade de armazenamento, Privacidade, backup e restauração, Fronteira de rede, CI/CD e implantação	QEX-04
delta	Escopo autoral, integração e atribuição	Nenhum novo defeito confirmado nos caminhos revisitados
fastapi-pagination	Escopo autoral, integração e atribuição	Nenhum novo defeito confirmado nos caminhos revisitados

Novos achados por tema

Confiança de backups e proteção de segredos

PROBLEMA OPS-05 - Importação interrompida após publicar o pacote perde as marcas de origem externa e proteção

- **Repositório:** containerops.
- **Categoria:** Segurança, backup e recuperação de falhas.
- **Gravidade:** Média.
- **Localização:** scripts/ops.py:1422-1431; scripts/backup_package.py:661-687; scripts/backup_catalog.py:350-360,415-425; scripts/ops.py:760-766.
- **Esforço:** médio.
- **Prioridade:** P1.

Problema identificado:

Importação interrompida após publicar o pacote perde as marcas de origem externa e proteção

Evidência:

execute_backups chama import_backup, que finaliza o diretório no destino, e só depois chama register_backup(imported=True, protected=True). Esses flags não estão duravelmente associados à publicação. A prova isolada simulou OSError no registro após a cópia/rename reais. discover reabriu o pacote como valid/imported=false/protected=false; restore_test sem --trusted-source atingiu o preflight. Ao registrar corretamente a mesma cópia, o controle foi recusado antes do preflight. Outra cópia posterior da mesma origem fez a órfã aparecer como candidata em plan_retention(keep_last=0). Recibo e script: _auditoria_2026-10-13/containerops-import-registration-repro.json e probe_import_registration.py.

Por que isso é um problema:

O fallback de compatibilidade para backups sem registro atribui imported=False/protected=False. Uma falha entre duas gravações independentes converte ausência de informação em confiança de origem local e ausência de proteção. A importação retorna erro, mas o comando seguinte ainda descobre e seleciona os bytes publicados.

Impacto:

Depois de falha de escrita/interrupção nessa janela, restore pode avançar sem o reconhecimento explícito de confiança exigido para pacotes importados; a cópia também pode entrar em plano de retenção. PostgreSQL alerta que restauração executa instruções escolhidas pela origem. O restore continua usando banco descartável e papéis/mounts definidos: não foi demonstrado escape de container, execução real de dump malicioso nem exclusão de backup.

Correção recomendada:

Fazer a classificação de origem e a proteção sobreviverem à interrupção entre a publicação do diretório e o catálogo. Enquanto um pacote importado não tiver registro concluído, tratá-lo conservadoramente como pendente/protegido e nunca presumir origem local. Recuperar o registro antes de liberar retenção e manter o reconhecimento explícito de confiança para restore.

Implementação sugerida:

Gravar um journal/intenção de importação no runtime, sob operation_lock, antes de publicar o diretório; incluir ID, hashes esperados e imported/protected=true. Após o rename, concluir o registro do catálogo e só então marcar a intenção como concluída. discover/select_backup devem reconhecer a intenção pendente e impedir que defaults imported=false/protected=false a substituam. Alternativa mais simples para o contrato atual: pacotes portáteis sem registro íntegro ficam desconhecidos ou exigem confiança/proteção por padrão até adoção explícita. Não inferir confiança de campos do manifesto recebido nem apenas do prefixo do nome. Preservar política explícita para backups legados sem manifesto e testar a recuperação.

Arquivos afetados: `scripts/ops.py`, `scripts/backup_package.py`, `scripts/backup_catalog.py`, `tests/test_backup_package.py`, `tests/test_backup_catalog.py`, `tests/test_operations.py`, `docs/security.md`, `docs/runbooks.md`.

Como validar: Injetar falha antes do rename, depois do rename/antes do catálogo e durante a gravação do catálogo. Reabrir o runtime em novo processo e provar que nenhum pacote importado fica imported=false/protected=false, que restore sem --trusted-source não passa ao preflight e que retenção não o seleciona. Reexecutar importação normal e recuperação para provar conclusão idempotente, proteção preservada, hashes intactos e compatibilidade com backups legados. Usar somente fixtures sintéticas, sem pg_restore real.

PROBLEMA LOJ-08 - Erro de configuração pode incluir partes de valores sensíveis no diagnóstico de inicialização

- **Repositório:** loja-assistente.
- **Categoria:** Privacidade e configuração.
- **Gravidade:** Baixa.
- **Localização:** backend/src/loja_assistente/config.py:11,13,17,21,55-67.
- **Esforço:** baixo; estimativa: 1-3 horas para ocultar entradas e cobrir o erro; até 0,5 dia com migração dos consumidores para SecretStr.
- **Prioridade:** P2.

Problema identificado:

Erro de configuração pode incluir partes de valores sensíveis no diagnóstico de inicialização

Evidência:

SettingsConfigDict não habilita hide_input_in_errors e database_url, session_secret e openai_api_key são strings comuns. O módulo instancia Settings() durante o import, antes dos handlers HTTP que sanitizam erros. Com _env_file=None, segredo de sessão sintético válido e COOKIE_SECURE=false fora de DEMO_MODE, str(ValidationError) incluiu input_value e o sufixo c-key-NOT-A-CREDENTIAL da chave sintética. A prova registrou synthetic_key_visible_in_str_exception=false e synthetic_key_suffix_visible=true: não houve exposição do marcador completo nem uso de credencial real.

Por que isso é um problema:

Os validadores corretamente impedem uma configuração insegura, mas a representação padrão da falha também pode reproduzir partes do dicionário de configuração. A sanitização de erros HTTP não cobre a construção de Settings no import.

Impacto:

Diagnósticos de startup coletados por terminal, CI ou plataforma de logs podem conter fragmentos de valores sensíveis quando a configuração falha. A reprodução demonstrou somente exposição parcial de marcador artificial; não demonstrou vazamento de uma chave utilizável, acesso público aos logs ou vazamento real no projeto.

Correção recomendada:

Ocultar valores de entrada nas mensagens de validação de Settings e modelar campos sensíveis com tipos que mascaram sua representação.

Implementação sugerida:

Adicionar hide_input_in_errors=True a SettingsConfigDict. Usar SecretStr para chave do provedor, segredo de sessão e URL com credenciais, adaptando os pontos de consumo para get_secret_value() somente onde necessário. Ocultar inputs continua necessário mesmo com SecretStr, pois a validação pode falhar sobre dados brutos. Manter mensagens com nome do campo e causa segura; não registrar model_dump() nem errors() com input incluído.

Arquivos afetados: `backend/src/loja_assistente/config.py`, `backend/tests/test_secure_config.py`, `backend/src/loja_assistente/database.py`, `backend/src/loja_assistente/auth/service.py`, `backend/src/loja_assistente/auth/admission.py`, `backend/src/loja_assistente/auth/routes.py`, `backend/src/loja_assistente/assistant/interpretation.py`.

Como validar: Criar testes com marcadores sintéticos distintos para cada campo sensível e configurações inválidas por cookie/endpoint. Assegurar que str(ValidationError) e um diagnóstico de startup controlado não contêm o marcador nem seus fragmentos identificáveis, e que a causa da rejeição continua compreensível. Após a migração para SecretStr, verificar os usos legítimos de conexão, HMAC, presença da chave e cliente do provedor sem chamada externa.

Concorrência e configuração de recursos

PROBLEMA LOJ-07 - Login permite concorrência de Argon2 incompatível com o limite de memória do contêiner

- **Repositório:** loja-assistente.
- **Categoria:** Autenticação e disponibilidade.
- **Gravidade:** Média.
- **Localização:** backend/src/loja_assistente/auth/service.py:18; backend/src/loja_assistente/auth/routes.py:68-70; backend/src/loja_assistente/database.py:13-24; backend/src/loja_assistente/auth/admission.py:16-18; compose.yaml:46.
- **Esforço:** médio; estimativa: 0,5-1 dia.
- **Prioridade:** P1.

Problema identificado:

Login permite concorrência de Argon2 incompatível com o limite de memória do contêiner

Evidência:

PasswordHasher() usa 65536 KiB por verificação no runtime inspecionado. O login faz SELECT e verifica a senha de forma síncrona antes de encerrar a Session, mantendo sua conexão. O engine permite 5 conexões mais 10 de overflow, e o runtime AnyIO disponibiliza 40 threads. A admissão limita 5 tentativas por conta e 30 por origem em 60 segundos, sem um semáforo global de verificações. Quinze identidades distintas cabem nesse allowance inicial. A prova isolada, com admissão e hasher substituídos, confirmou 15 chamadas simultaneamente no ponto de verificação; também verificou a retenção da conexão por Session após um SELECT em SQLite. Quinze verificações no custo configurado representam 960 MiB de memória Argon2, enquanto o backend declara mem_limit=768m.

Por que isso é um problema:

Controle de tentativas por janela não limita suficientemente o número de operações caras simultâneas. Os limites de banco/thread e a configuração do hash permitem um orçamento de trabalho maior que a memória total do serviço, antes de considerar Python, conexões e demais requisições.

Impacto:

Uma rajada de logins para contas distintas pode competir por memória, CPU e conexões, degradando o login e outras rotas. Sob pressão, pode haver falhas ou reinício do contêiner; isso é risco deduzido da configuração, não OOM observado. O Compose publica em loopback, a origem é validada e já há limites por conta/origem, o que restringe o contexto de exposição e reduz a severidade. Não foi alegado ataque remoto ao ambiente do usuário.

Correção recomendada:

Adicionar limite explícito de verificações de senha simultâneas, dimensionado com a memória do contêiner e o número de processos, mantendo a admissão durável já existente.

Implementação sugerida:

Adquirir um slot por processo antes do trabalho Argon2 e liberá-lo em finally; na falta de slot, retornar erro transitório controlado com Retry-After, sem fila ilimitada. Para o processo único atual, começar com um limite conservador como duas verificações e validar a margem de memória. Documentar processos × slots × memória por hash, incluindo o restante do serviço. Não reduzir os parâmetros de senha automaticamente para compensar a falta de controle de concorrência. Se a leitura do usuário for desacoplada da Session antes do hash, revalidar estado/permissões antes de criar a sessão autenticada.

Arquivos afetados: `backend/src/loja_assistente/auth/service.py`, `backend/src/loja_assistente/auth/routes.py`, `backend/src/loja_assistente/config.py`, `backend/tests/test_login_admission.py`, `compose.yaml`, `docs/security.md`.

Como validar: Com hasher barato bloqueável, disparar mais tentativas do que os slots e demonstrar que nunca há mais verificações ativas que o limite, que os excedentes recebem resposta transitória e que slots são liberados em sucesso e exceção. Preservar testes de admissão por conta/origem. Em ambiente descartável com Docker, medir posteriormente pico de memória e latência com o hash real e o número de workers previsto; não tratar a prova atual como esse ensaio.

PROBLEMA CYC-04 - Configuração aceita quota não finita e desativa a suspensão por tamanho

- **Repositório:** cycleops.
- **Categoria:** Configuração e limites de recursos.
- **Gravidade:** Baixa.
- **Localização:** apps/backend/src/cycleops/worker/main.py:95-99; worker/processor.py:119-135.
- **Esforço:** baixo.
- **Prioridade:** P2.

Problema identificado:

Configuração aceita quota não finita e desativa a suspensão por tamanho

Evidência:

Settings converte a variável com float e rejeita apenas quota <= 0. No probe, NaN, inf e 1e999 foram aceitos; maintain com tamanho simulado de 2048 MiB manteve quota.exceeded=False. O controle 1024 ativou a suspensão.

Por que isso é um problema:

Comparar qualquer tamanho finito com NaN ou infinito positivo não detecta estouro. Uma configuração inválida passa pelo fail-fast, apesar da mensagem prometer valor acima de zero.

Impacto:

Erro de configuração pode remover o limite operacional sem aviso de configuração. Não há entrada remota para alterar essa variável e não foi observado disco cheio.

Correção recomendada:

Rejeitar números não finitos ao ler CYCLEOPS_DB_QUOTA_MB, mantendo o erro de configuração e o código de saída já existentes.

Implementação sugerida:

```
import math

if not math.isfinite(quota) or quota <= 0:
    raise ConfigError("CYCLEOPS_DB_QUOTA_MB must be finite and above 0")
```

Acrescentar NaN, inf, -inf e 1e999 aos casos de configuração recusada, com IDs curtos nos testes.

Arquivos afetados: `apps/backend/src/cycleops/worker/main.py`, `apps/backend/tests/test_worker_main.py`.

Como validar: As quatro entradas não finitas devem lançar ConfigError antes de abrir conexão. Valores positivos finitos, incluindo 0.5 e 1024, devem continuar aceitos; 0 e negativos recusados.

Integridade e validação na persistência

PROBLEMA QEX-04 - Blob armazenado é reutilizado sem conferir a identidade SHA-256

- **Repositório:** quake-exposure.
- **Categoria:** Integridade de armazenamento e recuperação.
- **Gravidade:** Média.
- **Localização:** src/quakeexposure/ingestion/blobs.py:13-31; ingestion/projection.py:117; exposure/worker.py:184-189.
- **Esforço:** médio.
- **Prioridade:** P1.

Problema identificado:

Blob armazenado é reutilizado sem conferir a identidade SHA-256

Evidência:

BlobStore.get apenas descomprime o arquivo; put retorna cedo se o caminho existir. A projeção e o cálculo não revalidam o hash antes de consumir os bytes. No probe, um gzip B válido no caminho identificado por A foi aceito, inclusive após novo put(A).

Por que isso é um problema:

CRC do gzip verifica o próprio gzip, não a correspondência entre conteúdo, nome SHA-256 e registro control.capture. Uma troca por outro arquivo válido pode conservar proveniência A para conteúdo B.

Impacto:

Após cópia/restauração incorreta ou adulteração local, projeções/cálculos podem consumir bytes diferentes da captura registrada. Corrupção aleatória que quebra gzip pode ser detectada; o caso demonstrado é gzip válido com conteúdo errado. Não foi demonstrado acesso remoto ao volume.

Correção recomendada:

Verificar SHA-256 dos bytes descomprimidos contra control.capture antes de projetar/calcular. Validar também o arquivo existente no caminho rápido de put e recusar divergência com erro operacional explícito.

Implementação sugerida:

```
class BlobIntegrityError(ValueError):
    pass

def get(self, rel: str, *, expected_sha256: str) -> bytes:
    body = gzip.decompress((self.root / rel).read_bytes())
    if hashlib.sha256(body).hexdigest() != expected_sha256:
        raise BlobIntegrityError("blob diverge da captura registrada")
    return body
```

Em project_capture, passar capture.sha256. Em _load_grid, selecionar blob_path e sha256 e passar ambos ao armazenamento. Em put, quando o caminho existe, conferir o conteúdo com o hash já calculado antes de retornar. Tratar divergência como inconsistência do arquivo e registrar identificadores, sem payload; preservar o arquivo suspeito para investigação e refazer a captura por procedimento explícito. Atualizar os call sites e testes da mudança de assinatura.

Arquivos afetados: `src/quakeexposure/ingestion/blobs.py`, `src/quakeexposure/ingestion/projection.py`, `src/quakeexposure/exposure/worker.py`, `tests/unit (novos testes de BlobStore)`, `tests/integration/test_collector.py`, `tests/integration/test_exposure.py`.

Como validar: Gravar A, substituir por gzip B válido e exigir BlobIntegrityError em get e put(A). Confirmar leitura/reutilização de A intacto. Cobrir gzip truncado separadamente. Provar que projeção/cálculo não avança com hash divergente; a integração em PostgreSQL deve ser executada quando Docker estiver disponível.

PROBLEMA REC-08 - Campos textuais JSON aceitam NUL que a persistência não representa

- **Repositório:** gestao-recebiveis.
- **Categoria:** Validação de entrada e confiabilidade da API.
- **Gravidade:** Média.
- **Localização:** backend/src/gestao_recebiveis/schemas.py:19; backend/src/gestao_recebiveis/auth.py:64; backend/src/gestao_recebiveis/app.py:79.
- **Esforço:** médio; estimativa: Pequeno/médio: 0,5 dia.
- **Prioridade:** P2.

Problema identificado:

Campos textuais JSON aceitam NUL que a persistência não representa

Evidência:

Probe puro com o Python 3.13.15, Pydantic 2.13.5 e Psycopg 3.3.6 do venv do projeto: PaymentInput aceitou NUL em note/idempotency_key, CancelInput em reason e LoginInput em email. StrDumper do Psycopg recusou os quatro valores com DataError: PostgreSQL text fields cannot contain NUL (0x00) bytes. Nota/chave/e-mail seguem para colunas ou consultas textuais; motivo também segue para AuditEvent.details JSONB. app.py trata IntegrityError, OperationalError e PoolTimeout, mas não essa entrada inválida. Recibo: _auditoria_2026-10-13/recebiveis-nul-boundary-probe.json.

Por que isso é um problema:

Os limites de tamanho e tipos não garantem representabilidade no banco. O CSV já rejeita NUL explicitamente, mas os modelos JSON e a pesquisa textual não compartilham esse contrato.

Impacto:

Uma requisição com esse caractere passa pela validação e pode terminar como erro interno em vez de 422 com identificação do campo. Não foi executado HTTP/PostgreSQL nesta prova e não se demonstrou alteração de saldo; a conclusão sobre 500 deriva do caminho de persistência e dos handlers existentes.

Correção recomendada:

Rejeitar NUL nos campos que serão persistidos/consultados como texto ou JSONB, antes de executar o domínio.

Implementação sugerida:

Criar um validador reutilizável para strings persistidas e aplicá-lo à chave idempotente, nota, motivo e e-mail, além de q e metadados de upload quando alcançam PostgreSQL. Não remover NUL silenciosamente de chaves, nem normalizar senhas como efeito colateral. Retornar erro de campo estável; preservar Unicode válido e as quebras de linha permitidas pelo contrato. Um handler genérico de DataError não substitui a validação da entrada conhecida.

Arquivos afetados: `backend/src/gestao_recebiveis/schemas.py`, `backend/src/gestao_recebiveis/routes.py`, `backend/src/gestao_recebiveis/imports.py (filename)`, `backend/tests/test_field_matrix.py`, `backend/tests/test_api.py`.

Como validar: Exigir 422 para NUL em note, idempotency_key, reason, email, busca e filename persistido. No teste com PostgreSQL, confirmar que saldo, pagamentos e auditoria permanecem iguais após a recusa. Preservar casos positivos com acentos, emoji e texto multilinha permitido. Distinguir o teste unitário de schema/adaptação do teste integrado HTTP ainda necessário.

Observabilidade e correlação de erros

PROBLEMA EVD-07 - Erros rejeitados pelo limite de corpo apresentam dois IDs de rastreamento

- **Repositório:** evidencedesk.
- **Categoria:** Observabilidade e contrato de erro.
- **Gravidade:** Baixa.
- **Localização:** backend/src/evidencedesk/body_limit.py:28-43; backend/src/evidencedesk/app.py:73,107-112.
- **Esforço:** baixo; estimativa: 1-2 horas.
- **Prioridade:** P2.

Problema identificado:

Erros rejeitados pelo limite de corpo apresentam dois IDs de rastreamento

Evidência:

JsonBodyLimit cria um UUID próprio e o coloca no corpo e no cabeçalho de suas rejeições. O middleware externo request_context cria outro UUID, substitui X-Request-ID e usa esse segundo valor no evento http.completed. A prova evd_request_id_probe.json reproduziu HTTP 413 com JSON request_id=92ebc0322a3f4c7d89d12b771dee2661 e X-Request-ID=5f5dd49c9eb5405a9a140b393ac278b4. A mesma construção de rejeição atende tamanho declarado/recebido, Content-Length inválido e timeout.

Por que isso é um problema:

O ID que a interface recebe no envelope de erro não é a chave de correlação registrada pelo middleware externo. Os testes do limitador isolado não exercitam essa composição.

Impacto:

Um relato de erro com o ID do JSON pode não localizar o evento HTTP correspondente nos logs, dificultando o diagnóstico de falhas de envio. Não foi identificado vazamento de dados nem desvio do limite de tamanho.

Correção recomendada:

Definir um único request_id por requisição e reutilizá-lo no limitador, no envelope JSON, no cabeçalho e no evento de conclusão.

Implementação sugerida:

Em JsonBodyLimit, obter o ID de scope['state'] já inicializado pelo middleware externo; criar e guardar um fallback somente se não existir. Manter essa regra válida quando o limitador for usado isoladamente. Evitar gerar um novo UUID dentro do ramo de rejeição.

Arquivos afetados: `backend/src/evidencedesk/body_limit.py`, `backend/src/evidencedesk/app.py`, `backend/tests/unit/test_body_limit.py`.

Como validar: Adicionar testes da aplicação criada por create_app, além do middleware isolado. Para rejeição por tamanho declarado, tamanho recebido, Content-Length inválido e prazo controlado, afirmar que o ID do JSON, do cabeçalho e do evento http.completed é igual, não vazio e gerado pelo servidor. Usar timeout curto controlado no teste.

Documentação de migração e retenção

PROBLEMA REC-07 - O roteiro de atualização descreve a migração atual como apenas aditiva

- **Repositório:** gestao-recebiveis.
- **Categoria:** Documentação operacional e migrações.
- **Gravidade:** Média.
- **Localização:** docs/security.md:32; backend/migrations/versions/e71c2f_partial_payments.py:14.
- **Esforço:** baixo; estimativa: Pequeno: 1-2 horas.
- **Prioridade:** P2.

Problema identificado:

O roteiro de atualização descreve a migração atual como apenas aditiva

Evidência:

O passo 4 de Atualizar uma instalação existente sem apagar dados afirma que a migração acrescenta somente login_admission. No HEAD, alembic upgrade head também alcança e71c2f: bloqueia receivables/payments, valida saldos e quitação legada, remove a unicidade payments.receivable_id e acrescenta índice e constraint de valor positivo. Dados incompatíveis provocam recusa explícita. O downgrade também rejeita múltiplos recebimentos ou pagamento parcial.

Por que isso é um problema:

A instrução é apresentada como procedimento atual, mas descreve o escopo da revisão anterior d6420a. O operador não é informado de que haverá validação e mudança do contrato financeiro.

Impacto:

Uma atualização pode parar pela guarda de consistência ou aguardar locks de tabelas sem que o roteiro explique a causa e a resposta esperada. Não foi demonstrada perda de dados; as próprias guardas são uma qualidade do código.

Correção recomendada:

Atualizar o procedimento para identificar a revisão financeira, suas condições de recusa e os limites de retorno ao schema anterior.

Implementação sugerida:

Substituir a última frase do passo 4 por: "O comando aplica as revisões pendentes até o head. A revisão e71c2f valida os dados financeiros legados sob lock de receivables e payments, retira a unicidade de pagamento por título e acrescenta índice e regra de valor positivo. Se houver inconsistência, a migração para sem corrigi-la silenciosamente; preserve o backup, examine o título indicado e reconcilie a origem antes de repetir." Acrescentar que downgrade não apaga parcelas para caber no modelo antigo e pode ser recusado. Preservar os passos já existentes de parada dos escritores, backup e validação em banco separado.

Arquivos afetados: `docs/security.md`, `docs/verification.md (conferir alinhamento)`, `backend/migrations/versions/e71c2f_partial_payments.py (referência, sem alteração necessária)`.

Como validar: Confrontar o roteiro com alembic history e o arquivo e71c2f; em ambiente descartável, validar upgrade de quitação legada consistente, recusa de saldo incompatível e recusa de downgrade após parcelas. As duas últimas são resultados esperados de proteção.

PROBLEMA CYC-05 - Documentação apresenta a limpeza do arquivo bruto como teto rígido de armazenamento

- **Repositório:** cycleops.
- **Categoria:** Documentação operacional e retenção.
- **Gravidade:** Baixa.
- **Localização:** SECURITY.md:11; apps/backend/src/cycleops/worker/archive.py:1-2,51-60,67-79,98.
- **Esforço:** baixo.
- **Prioridade:** P2.

Problema identificado:

Documentação apresenta a limpeza do arquivo bruto como teto rígido de armazenamento

Evidência:

SECURITY.md afirma teto de 500 MB. ArchiveWriter chama prune apenas em _open, na rotação/reinício; cada write posterior apenas escreve e faz flush. prune preserva keep mesmo acima do máximo; test_worker_archive.py:98-99 testa essa exceção explicitamente.

Por que isso é um problema:

O limite orienta a remoção dos arquivos anteriores durante a rotação. Ele não é uma garantia de espaço máximo, nem uma purga periódica independente de novas escritas. O ativo pode crescer entre duas rotações.

Impacto:

Quem dimensionar o volume supondo um teto rígido pode reservar espaço insuficiente. Não foi medido consumo real de 500 MiB; o achado é a promessa mais forte que o comportamento intencional.

Correção recomendada:

Corrigir SECURITY.md e a docstring para explicar a política real. Manter a preservação do arquivo ativo; um teto físico exigiria decisão separada de rotação por tamanho e comportamento quando não houver espaço.

Implementação sugerida:

Texto para substituir apenas a descrição do arquivo bruto:

"O arquivo bruto do live é dividido por hora. Ao abrir um novo arquivo, a limpeza remove arquivos anteriores com última escrita há mais de 48 horas e depois os mais antigos enquanto o conjunto ultrapassa 500 MiB. O arquivo em uso é preservado; por isso o volume pode passar desse valor entre rotações."

Docstring correspondente: "Hourly NDJSON.gz archive. On rotation, prune older files by a 48-hour retention target and a 500 MiB size threshold, preserving the active file; this is not a hard disk quota."

Arquivos afetados: `SECURITY.md`, `apps/backend/src/cycleops/worker/archive.py`.

Como validar: Conferir que a descrição identifica momento da limpeza, unidade MiB e exceção do arquivo ativo. Preservar os testes existentes e o probe com limiar reduzido; não rodar teste de carga contra o feed real.

Precisão adicional de uma correção anterior

VDP-02 - publicação atômica do HTML. Ao substituir a escrita direta por arquivo temporário e `os.replace`, preservar/definir antes da troca as permissões de leitura necessárias ao report-server, que roda como UID 65534. Um temporário criado por `mkstemp` normalmente nasce com 0600; a atomicidade não deve tornar o resultado ilegível para o servidor. Validar um GET sob o usuário do container após a troca, além da integridade do documento. Isso complementa a recomendação anterior; não é um novo defeito já existente nem autoriza ampliar permissões indiscriminadamente.

Controles observados e verificações por repositório

arthurjoanes

Versão examinada: `603c69d889f19d0781f486238a55e242ca55b66e`.

O que foi confirmado ou encontrado implementado:

- PER-01 continua sendo a mudança objetiva de descoberta das demos; não foi criado novo achado para repetir esse ponto. Tradução preserva o sentido das experiências declaradas.

Verificações desta rodada:

• **Procedimento:** Releitura PT/EN e inventário GitHub **Resultado:** 11 repositórios públicos permanecem no inventário; propósito e alegações profissionais correspondem nas duas versões. **Limite:** Não valida métricas do empregador ou trajetória pessoal.

Arquivos e trechos adicionais examinados:

• `README.md`

• `README.en.md`

• `inventário público atualizado`

Limitações restantes:

• Perfil não tem superfície de execução para auditar sessão, banco ou deploy. Não há base para contestar métricas privadas como falsas.

evidencedesk

Versão examinada: `e4dd8efe73e50999b9a75896ce74b75d460223da`.

O que foi confirmado ou encontrado implementado:

• O contexto autenticado deriva tenant e usuário da sessão e verifica usuário/tenant habilitados. As mutações verificam Origin e CSRF; cookies usam HttpOnly e SameSite=Lax, e a configuração exige proteção adicional fora do modo local.

• A leitura do banco combina contexto de tenant por transação, políticas RLS e verificações explícitas de coleção/incidente. A prontidão verifica o papel do banco contra privilégios incompatíveis com o isolamento; isso foi inspecionado, não exercitado em PostgreSQL nesta rodada.

• O login limita verificações Argon2 simultâneas a duas, além da admissão durável por identidade/origem. Uploads têm limites por processo e tenant, tamanho, hash de manifesto e prazo. O parser isolado aplica limites de recursos em Linux e limites de linhas/saída; a execução Windows não demonstra RLIMIT do contêiner Linux.

• A publicação de objetos usa arquivo temporário, sincronização e instalação imutável; a limpeza mantém estados duráveis e verifica referências/produtores antes de remover o arquivo e devolver quota. O protocolo de restauração inclui ledger de exclusões e falha fechada para condições incompatíveis.

• Logs e telemetria usam campos limitados, sem registrar conteúdo dos documentos, prompts ou mensagens brutas de exceções nos caminhos revisados. Rótulos de métricas limitam método e rota.

• A liberação de modelo verifica endpoint permitido, prompt/esquema/modelo e limites; o adaptador desabilita armazenamento do provedor e retries automáticos. O despacho exige reserva de orçamento e preserva consumo desconhecido. Os 38 testes executados reforçam o armazenamento privado e a validação de release.

• O workflow declara permissões de leitura, fixa ações por SHA, desabilita persistência de credenciais do checkout e verifica dependências/segredos/imagens. Foram lidas as etapas; sua existência não foi tratada como prova de execução recente ou ausência de vulnerabilidades.

Verificações desta rodada:

• **Procedimento:** git -C evidencedesk rev-parse HEAD; git -C evidencedesk status --short **Resultado:** HEAD e4dd8efe73e50999b9a75896ce74b75d460223da; árvore limpa. **Limite:** Leitura local. A comparação com origin HEAD pertence à primeira rodada.

• **Procedimento:** python -B _auditoria_2026-10-13/evd_request_id_probe.py, com PYTHONPATH da cópia execution/evd-domain/backend/src, ED_AI_PROVIDER=disabled e sem chave Azure **Resultado:** Uma requisição de 512 KiB + 1 byte ao login retornou HTTP 413/request**oo_larg** O request_id do JSON foi 92ebc0322a3f4c7d89d12b771dee2661; o cabeçalho X-Request-ID foi 5f5dd49c9eb5405a9a140b393ac278b4.

Resultado salvo em evd_request_id_probe.json. **Limite:** TestClient em processo, sem lifespan, banco, rede, Docker ou carga. Confirma a divergência JSON/cabeçalho na pilha real de middleware; a correspondência do cabeçalho com o campo de log é leitura de app.py.

• **Procedimento:** Na cópia execution/evd-domain: python -B -m pytest backend/tests/unit/test_private_storage.py backend/tests/unit/test_model_release.py -p no:cacheprovider -q **Resultado:** 38 testes passaram em 1,56 s. Fornecedor de IA desabilitado; PYTHONDONTWRITEBYTECODE=1. **Limite:** Runtime Python já existente no workspace; testes unitários com dependências locais, sem serviço Azure, PostgreSQL, Docker ou execução do workflow de CI.

Arquivos e trechos adicionais examinados:

- `backend/src/evidencedesk/identity/service.py`
- `backend/src/evidencedesk/identity/routes.py`
- `backend/src/evidencedesk/identity/admission.py`
- `backend/src/evidencedesk/app.py`
- `backend/src/evidencedesk/config.py`
- `backend/src/evidencedesk/body_limit.py`
- `backend/src/evidencedesk/database.py`
- `backend/src/evidencedesk/telemetry/signals.py`
- `backend/src/evidencedesk/ingestion/limits.py`
- `backend/src/evidencedesk/ingestion/parser.py`
- `backend/src/evidencedesk/ingestion/routes.py`
- `backend/src/evidencedesk/ingestion/service.py`
- `backend/src/evidencedesk/ingestion/contracts.py`
- `backend/src/evidencedesk/evidence/storage.py`
- `backend/src/evidencedesk/evidence/routes.py`
- `backend/src/evidencedesk/evidence/service.py`
- `backend/src/evidencedesk/evidence/deletion.py`
- `backend/src/evidencedesk/retention/service.py`
- `backend/src/evidencedesk/retention/cleanup.py`
- `backend/src/evidencedesk/administration/routes.py`
- `backend/src/evidencedesk/deletion_ledger.py`
- `backend/src/evidencedesk/jobs/service.py`
- `backend/src/evidencedesk/worker.py`
- `backend/src/evidencedesk/investigations/streams.py`
- `backend/src/evidencedesk/investigations/routes.py`
- `backend/src/evidencedesk/investigations/budget.py`
- `backend/src/evidencedesk/model_runtime/release.py`
- `backend/src/evidencedesk/model_runtime/azure.py`
- `backend/src/evidencedesk/maintenance.py`

- backend/migrations/versions/0007_retention_ownership.py
- backend/pyproject.toml
- .github/workflows/ci.yaml
- docs/api-contract.md
- docs/runbooks/backup-restore.md
- backend/tests/unit/test_body_limit.py
- backend/tests/unit/test_private_storage.py
- backend/tests/unit/test_model_release.py
- backend/tests/integration/test_maintenance.py

Limitações restantes:

- Leitura dirigida: arquivos extensos de jobs, budget, retenção, limpeza, Azure, manutenção e runbooks foram inspecionados por funções/seções relevantes; files_read não significa revisão linha a linha de todos esses arquivos.
- Sem PostgreSQL ou Docker nesta rodada: não foram reexecutados isolamento RLS, integração de sessões, fluxo de upload completo, limpeza física, restauração ou concorrência real entre processos.
- Nenhuma chamada a fornecedor de IA. A análise do orçamento verifica a implementação local e não garante equivalência com uma fatura externa.
- Não houve novo scan de vulnerabilidades de dependências nem consulta individual a todas as versões. Nenhuma ausência de CVE foi inferida do workflow.
- A manutenção trata produtores ativos de forma conservadora. Sem reproduzir um caso operacional adicional, isso não foi promovido a novo defeito de retenção.
- Nenhum bypass de autenticação ou vazamento entre tenants foi confirmado na revisão dirigida; isso não equivale a uma certificação de segurança.

Referências adicionais:

- Código local e prova de composição de middleware

gestao-recebiveis

Versão examinada: 7477a6afe0bbc0a5308cec120f04a5f7384b81ad.

O que foi confirmado ou encontrado implementado:

- Chaves idempotentes são coordenadas mesmo quando requisições apontam para títulos diferentes; o conteúdo canônico impede reutilização com outro pedido.
- Recebimentos usam saldo calculado sob lock do título e guardas para valor positivo/limite do saldo; cancelamento com parcela é recusado.
- O resultado desconhecido do provedor preserva a autorização original; lease_token impede finalização por posse antiga e o dia comercial tem guarda própria.
- A migração financeira confere invariantes e recusa downgrade incompatível, em vez de apagar ou reajustar dinheiro.
- Admissão de login é persistida em transação separada, limita Argon2 e guarda HMACs de identidade/origem em vez dos valores crus.
- Escopo de uma empresa é explícito. Não classifiquei ausência de tenant como falha nem contas fictícias publicadas como vazamento de credenciais reais.

Verificações desta rodada:

• **Procedimento:** git status --short; git rev-parse HEAD (início e fim da rodada) **Resultado:** HEAD permanece 7477a6afe0bbc0a5308cec120f04a5f7384b81ad; checkout limpo no início e no fim desta rodada. **Limite:** Não houve novo fetch nem alteração do remoto.

• **Procedimento:** Revisão dirigida de pagamento, importação, auth, worker e relatórios **Resultado:** Lock de chave idempotente antecede lock do título; quitação e cancelamento coordenam reminders; importação usa savepoint; relatórios usam REPEATABLE READ independente da sessão de autenticação. Origin/CSRF/perfil são verificados nas mutações. Datas extremas são tratadas por diferença de datas/clamp, sem somar dias fora do domínio. **Limite:** Leitura não equivale a reexecutar corridas PostgreSQL. Nenhuma quebra nova de saldo ou de autorização foi demonstrada.

• **Procedimento:** Confronto docs/security.md com migrações d6420a/e71c2f e env.py **Resultado:** Confirmado REC-07: roteiro atual diz alteração apenas aditiva de login_admission, mas head inclui migração de contrato de pagamentos com locks e validações. **Limite:** A migração financeira não foi executada nesta rodada; o conflito é documental e verificável diretamente na fonte.

• **Procedimento:** Probe puro de schemas + psycopg.types.string.StrDumper, Python 3.13.15 do venv existente; PYTHONDONTWRITEBYTECODE=1 **Resultado:** Quatro campos com NUL aceitos pelos schemas e recusados pelo adaptador com DataError. Versões registradas: Pydantic 2.13.5 e Psycopg 3.3.6. Recibo: recebiveis-nul-boundary-probe.json. database_accessed=false. **Limite:** Não é teste HTTP integrado nem acesso ao PostgreSQL. Motivo de cancelamento usa JSONB no caminho real; a restrição de JSONB foi confirmada na documentação PostgreSQL 18.

• **Procedimento:** Revisão de prova de restore, permissões, CI e containers **Resultado:** Runner separa evidência privada/publicável, aplica/verifica ACL antes dos segredos, restringe namespaces e destinos e mantém checksums sem alegar autenticidade. Runtime PostgreSQL separado do owner; scripts de CI usam credenciais temporárias e ambientes de teste. **Limite:** Sem executar Docker, migrações, restore, scan ou serviços reais. Hash local não autentica backup, limite já explicitado no projeto e não tratado como achado novo.

Arquivos e trechos adicionais examinados:

• `backend/src/gestao_recebiveis/receivables.py`

• `backend/src/gestao_recebiveis/auth.py`

• `backend/src/gestao_recebiveis/routes.py`

• `backend/src/gestao_recebiveis/schemas.py`

• `backend/src/gestao_recebiveis/clock.py`

• `backend/src/gestao_recebiveis/database.py`

• `backend/src/gestao_recebiveis/balances.py`

• `backend/src/gestao_recebiveis/import_csv.py`

• `backend/src/gestao_recebiveis/imports.py`

• `backend/src/gestao_recebiveis/login_admission.py`

• `backend/src/gestao_recebiveis/config.py`

• `backend/src/gestao_recebiveis/reminders/policy.py`

• `backend/src/gestao_recebiveis/reminders/service.py`

• `backend/src/gestao_recebiveis/reminders/provider.py`

• `backend/src/gestao_recebiveis/worker.py`

• `backend/src/gestao_recebiveis/app.py`

- backend/src/gestao_recebiveis/request_limits.py
- backend/src/gestao_recebiveis/models.py
- backend/src/gestao_recebiveis/reporting.py
- backend/src/gestao_recebiveis/provision_database.py
- backend/src/gestao_recebiveis/audit.py
- backend/migrations/versions/e71c2f_partial_payments.py
- backend/migrations/versions/d6420a_login_admission.py
- backend/migrations/env.py
- backend/tests/test_reports.py
- backend/tests/test_field_matrix.py
- backend/tests/test_database_roles.py
- backend/tests/restore_support.py
- scripts/verify_restore_backup.py
- scripts/prove_restore.py
- backend/pyproject.toml
- backend/Dockerfile
- frontend/Dockerfile
- frontend/Dockerfile.e2e
- frontend/next.config.ts
- database/Dockerfile
- compose.yaml
- compose.e2e.yaml
- .dockerignore
- .github/workflows/ci.yml
- .github/workflows/secrets.yml
- docs/security.md
- docs/demo.md (trechos)

Limitações restantes:

- Não houve validação end-to-end de todos os modelos ou execução da suíte PostgreSQL; os testes existentes foram inspecionados por recorte.
- REC-08 tem reprodução do limite schema/adaptador e inferência controlada sobre o caminho HTTP; não se declara 500 observado em servidor nem corrupção financeira.
- Nomes de arquivos/metadados e busca foram identificados como campos a cobrir na correção; a prova executada contém somente os quatro campos discriminados.
- Não houve scan atual de CVEs nem instalação limpa dos lockfiles; nenhuma afirmação de dependência vulnerável foi acrescentada.
- Arquivos extensos de runner e testes foram lidos em trechos dirigidos. Nenhuma gravação em repositório foi autorizada ou realizada.

Referências adicionais:

- PostgreSQL 18 - JSON Types
- Psycopg 3 - Adapting basic Python types

loja-assistente

Versão examinada: `075ea83a0a7b99ccef5868050609c3c36c4d907f`.

O que foi confirmado ou encontrado implementado:

- As consultas analíticas usam operações previamente definidas e filtros de tenant/loja; o cliente não fornece SQL livre. Histórico, favoritos e retomada verificam permissões atuais, com tratamento de escopo revogado nos caminhos revisados.
- Sessões usam token aleatório de 48 bytes e digest HMAC no banco. Mutações verificam Origin/CSRF; cookies usam HttpOnly e SameSite=Lax, com Secure exigido fora de DEMO_MODE. O segredo fora de demo deve ter pelo menos 32 caracteres.
- O proxy encaminha somente o cookie da aplicação e não confia em X-Forwarded-For; o backend desabilita proxy headers e o Compose publica serviços em loopback. A arquitetura usa proxy de mesma origem; ausência de CORS permissivo não é um defeito.
- A admissão de login é durável, por conta e origem, oferece Retry-After e usa identificadores derivados em lugar de e-mail/IP brutos. Sucesso libera sua reserva e falha conserva a contagem. Esse controle é útil, embora não substitua o limite de concorrência do LOJ-07.
- O limite de corpo conta chunks e rejeita acima de 16 KiB. O proxy aplica prazo de 45 segundos que abrange a leitura do pedido e encaminhamento, com propagação de cancelamento.
- O handler de erro HTTP registra tipo de exceção, sem texto bruto, e os traces do provedor contêm metadados limitados, hashes e uso, sem prompt/resposta completa. O adaptador usa store=false e desabilita retries automáticos; o LOJ-08 se limita ao diagnóstico anterior à inicialização da aplicação.
- A reserva de orçamento e a marcação de despacho usam transações duráveis; custo desconhecido não é liberado como se o pedido fosse gratuito. Sem chamada externa nesta revisão, não se alegou equivalência garantida com a fatura do fornecedor.
- O workflow usa permissões reduzidas, ações por SHA, checkout sem credenciais persistidas, Gitleaks com verificação de checksum e saída redigida e scanner de imagem fixado por digest. Docker usa imagens fixadas, instalação congelada e usuário sem root. A análise é da configuração, sem nova execução da CI.

Verificações desta rodada:

- **Procedimento:** git -C loja-assistente rev-parse HEAD; git -C loja-assistente status --short **Resultado:** HEAD 075ea83a0a7b99ccef5868050609c3c36c4d907f; árvore limpa. **Limite:** Leitura local. A comparação com origin HEAD pertence à primeira rodada.
- **Procedimento:** python -B _auditoria_2026-10-13/loja_login_capacity_probe.py, com PYTHONPATH da cópia execution/loj-domain/backend/src, DEMO_MODE=true, LLM_ENABLED=false e chave de provedor vazia **Resultado:** Hasher configurado com memory**ost=655** KiB, time_cost=3 e parallelism=4; pool de 5+10 conexões; runtime AnyIO com 40 tokens; admissão de origem=30 e conta=5. Quinze chamadas de login com verificador barato substituto alcançaram simultaneamente a verificação e terminaram em 401. Uma Session SQLAlchemy sobre SQLite manteve conexão após SELECT e a liberou após rollback. Orçamento calculado: 15 × 64 MiB = 960 MiB, contra mem_limit=768m no Compose. **Limite:** A admissão e o trabalho de senha foram substituídos por stubs na prova de concorrência. Não foram executados Argon2 pesado em paralelo, HTTP de carga, PostgreSQL, swap ou OOM. SQLite demonstra o ciclo de vida do ORM, não mede o pool PostgreSQL. Quinze identidades distintas cabem nos limites iniciais declarados por conta/origem; isso é análise do orçamento permitido, não uma medição de RSS.

• **Procedimento:** python -B _auditoria_2026-10-13/loja_config_redaction_probe.py, com _env_file=None e marcador sintético audit-synthetic-key-NOT-A-CREDENTIAL **Resultado:** Configuração inválida por COOKIE**ECURE=f**se fora de DEMO_MODE produziu ValidationError. str(error) incluiu input_value com o sufixo c-key-NOT-A-CREDENTIAL. O marcador completo não apareceu, devido à abreviação do Pydantic. hide_input_in_errors estava false. Resultado salvo em loja_config_redaction_probe.json. **Limite:** Nenhum arquivo .env real, chave real, serviço de log ou provedor foi usado. A prova verifica a representação da exceção; a chegada a stderr na inicialização normal decorre da construção não capturada de Settings() no import, não de uma execução de deploy.

Arquivos e trechos adicionais examinados:

- `backend/src/loja_assistente/auth/service.py`
- `backend/src/loja_assistente/auth/routes.py`
- `backend/src/loja_assistente/auth/admission.py`
- `backend/src/loja_assistente/config.py`
- `backend/src/loja_assistente/app.py`
- `backend/src/loja_assistente/database.py`
- `backend/src/loja_assistente/request_limits.py`
- `backend/src/loja_assistente/observability/routes.py`
- `backend/src/loja_assistente/assistant/provider_trace.py`
- `backend/src/loja_assistente/assistant/interpreters/openai_adapter.py`
- `backend/src/loja_assistente/assistant/budget.py`
- `backend/src/loja_assistente/assistant/budget_policy.py`
- `backend/src/loja_assistente/assistant/interpretation.py`
- `backend/src/loja_assistente/conversations/service.py`
- `backend/src/loja_assistente/favorites/service.py`
- `backend/src/loja_assistente/clarifications/service.py`
- `backend/src/loja_assistente/analytics/queries.py`
- `backend/src/loja_assistente/analytics/service.py`
- `backend/Dockerfile`
- `compose.yaml`
- `frontend/src/app/api/[...path]/route.ts`
- `.github/workflows/ci.yml`
- `docs/security.md`
- `docs/api-contract.md`
- `docs/decisoes-tecnicas.md`
- `backend/tests/test_login_admission.py`
- `backend/tests/test_request_limits.py`
- `backend/tests/test_secure_config.py`
- `backend/tests/test_budget_adapter.py`

Limitações restantes:

• Leitura dirigida: budget, clarifications, consultas analíticas, documentação e alguns testes foram revisados em seções específicas; files_read não implica revisão integral de cada arquivo extenso.

• Provas usaram o Python já disponível no workspace com o código copiado do HEAD; não constituem instalação limpa do lockfile da Loja Assistente nem teste da imagem publicada.

• Sem PostgreSQL/Docker, nenhum ensaio de carga real, nenhuma medição de RSS e nenhum Argon2 caro em paralelo. Os números do LOJ-07 descrevem configuração, concorrência de stubs e orçamento permitido.

• Sem chamadas a LLM, serviço externo de logs ou uso de segredo real. LOJ-08 demonstrou somente um fragmento de marcador sintético na representação da exceção.

• Não foi encontrada rotina automática geral de retenção para sessões/conversas nos caminhos revisados; a documentação já trata retenção como requisito de evolução para produção. Sem requisito de prazo atual descumprido, isso fica como limite de implantação, não como novo defeito confirmado.

• O backend direto não demonstrou nesta rodada um prazo de leitura equivalente ao proxy. A publicação local em loopback e o prazo do proxy são contexto essencial; não se alegou exposição pública nem ataque executado.

• Não houve novo scan de dependências ou revisão de todas as CVEs. O workflow não foi usado como prova de ausência de vulnerabilidades.

• Nenhum bypass de sessão, CSRF ou vazamento entre tenants foi confirmado nos fluxos dirigidos. Não houve teste completo de integração do sistema nesta rodada.

Referências adicionais:

• argon2-cffi - How to Hash a Password

• SQLAlchemy - Session Basics

• Starlette - Thread Pool

• Pydantic - ConfigDict.hide_input_in_errors

• Prova isolada de orçamento de login

• Prova de redação de erro de configuração

varejo-data-pipeline

Versão examinada: `cd8bff90ce9d1ef37f727ec6f7fd9d8cbe615134`.

O que foi confirmado ou encontrado implementado:

• A reconstrução do candidato lê o histórico oficialmente publicado, evitando adotar automaticamente restos de candidato anterior que falhou.

• A validação não confunde reconciliação de receita com integridade da venda: SALE_DATE_CONFLICT verifica datas dos itens ativos.

• No adapter OpenLineage, um manifesto de candidato isolado não basta para provar commit; a reconstrução usa ponteiro/ancestralidade e registros legados explícitos.

• A retenção é conservadora e declara que não autoriza VACUUM ou exclusão individual de arquivos Delta.

• As ferramentas JVM checam hashes upstream/rebuild/provenance e detectam ZIP com nomes duplicados, sem tratar o scanner como assinatura de confiança.

Verificações desta rodada:

- **Procedimento:** git status --short; git rev-parse HEAD (início e fim da rodada) **Resultado:** HEAD permanece cd8bff90ce9d1ef37f727ec6f7fd9d8cbe615134; checkout limpo no início e no fim desta rodada. **Limite:** A comparação remota foi realizada na primeira rodada; nesta rodada apenas a identidade local foi novamente consultada.
- **Procedimento:** Leitura dirigida de publicador, pipeline, transformações e contratos **Resultado:** Escritor único usa flock; os candidatos partem do histórico da publicação oficial; conflitos e separação de data da mesma venda bloqueiam publicação. O ponto visível usa JSON temporário, fsync, replace e fsync do diretório. Valores/timestamps têm validação e canonicalização explícitas. **Limite:** Revisão estática; não reexecuta Spark/Delta ou concorrência em Linux.
- **Procedimento:** Leitura de proof_contracts.py, prove_state.py e lineage.py **Resultado:** Backup verifica inventário antes/depois, recusa entradas especiais e destino ocupado. Retenção protege diretórios Delta completos e perfis com hash/identidade. Reexportação de lineage usa lock e cadeia de publicações comprometidas, com eventos imutáveis. **Limite:** VDP-01 já cobre a ordenação por mtime pós-restauração; não duplicado. Alguns trechos do runner grande foram lidos de forma dirigida.
- **Procedimento:** Leitura de Compose, workflows e instaladores de JAR **Resultado:** Batch sem rede, mounts de código somente leitura, recursos limitados; componentes baixados têm hashes; rebuilds são validados em conjunto antes de copiar; CI faz scan e exige presença dos inventários OS/JVM/Python no gate. **Limite:** Não houve build nem scan novo. Existência de gate não certifica ausência atual de vulnerabilidades; não foi feita avaliação exaustiva de todos os backports.

Arquivos e trechos adicionais examinados:

- `src/retail_pipeline/publication.py`
- `src/retail_pipeline/pipeline.py`
- `src/retail_pipeline/transformations.py`
- `src/retail_pipeline/contracts.py`
- `src/retail_pipeline/ingestion.py`
- `src/retail_pipeline/batch_contracts.py`
- `src/retail_pipeline/input_limits.py`
- `src/retail_pipeline/lineage.py`
- `scripts/proof_contracts.py`
- `scripts/prove_state.py`
- `scripts/fetch_artifact.py`
- `scripts/download_jars.py`
- `scripts/install_rebuilt_jars.py`
- `scripts/patch_runtime_jars.py`
- `scripts/check_image_scan.py`
- `compose.yaml`
- `.dockerignore`
- `.github/workflows/ci.yml`
- `.github/workflows/secrets.yml`

Limitações restantes:

• Nenhum achado novo confirmado no recorte desta rodada; isso não cancela VDP-01 a VDP-04 nem afirma ausência de falhas.

• Runtime declarado Linux/Spark/Delta não foi executado; daemon Docker anteriormente indisponível não foi iniciado.

• O produto é local e não oferece multiusuário/tenant; não tratei essa limitação explícita como vulnerabilidade.

• Não examinei exaustivamente toda a árvore vendor/backports ou todas as dependências transitivas.

• A futura correção de VDP-02 deve preservar permissões de leitura do report-server UID 65534: temporário mkstemp padrão 0600 seguido de replace precisa de política de modo apropriada. É cuidado de implementação do achado anterior, não defeito novo.

api-sentinel

Versão examinada: `a0e1375bfecabcdf0a5e98802598bff63bbe44f6`.

O que foi confirmado ou encontrado implementado:

• Não foi confirmado novo defeito de autenticação, autorização, SQL injection, SSRF ou divulgação de segredo nos caminhos examinados. Essa frase se limita à cobertura descrita; os achados API-01 a API-05 da primeira rodada continuam válidos.

• Credencial expirada/revogada é consultada no banco a cada autenticação. Limites de autenticação e trabalho em andamento precedem a operação de negócio; cancelamento libera tarefas e recursos.

• O log de exceção da API registra tipo e localização de frames, sem mensagem, locals ou código-fonte. Respostas de validação/erro são genéricas; não repassam SQL, payload ou credencial.

• ERP/probe limitam tempo e tamanho de resposta, recusam compressão inesperada e não seguem redirecionamentos. A ferramenta de contratos usa transporte sem proxy de ambiente, não segue redirects e limita o corpo.

• Papéis de banco e ACL Redis estão separados por função. Senhas PostgreSQL literais são credenciais locais explicitamente documentadas; não foram tratadas como vazamento de produção.

• CI usa escopo contents:read, referências SHA das Actions, fluxo específico de scan aplicação/proxy e limpeza de projetos identificados. Gitleaks tem exceções de caminhos/padrões delimitados; não há exclusão geral de evidências.

• Compose publica os serviços em loopback. Algumas imagens auxiliares são tags versionadas sem digest; isso foi registrado como limite de reprodutibilidade/escopo, sem alegar vulnerabilidade concreta ou inventar finding.

Verificações desta rodada:

• **Procedimento:** git status --short; git rev-parse HEAD; git ls-remote origin HEAD **Resultado:** Status vazio no início e ao encerrar; HEAD local e remoto iguais a a0e1375bfecabcdf0a5e98802598bff63bbe44f6. **Limite:** Não demonstra execução atual de CI, scanner ou da stack.

• **Procedimento:** Na cópia git archive da rodada anterior, com PYTHONDONTWRITEBYTECODE=1/PYTHONPATH=src e Python de .runtime/verifyvenv: python -X utf8 -m pytest tests/unit/test_auth_admission.py tests/unit/test_middleware.py tests/unit/test_config.py tests/unit/test_receiver.py tests/unit/test_receiver_config.py tests/unit/test_receiver_probe.py tests/unit/test_redis_bootstrap.py tests/unit/test_rate_limit.py tests/unit/test_review_cleanup.py tests/contracts/test_verifier.py -q -p no:cacheprovider **Resultado:** 161 passed, 4 skipped, 6 subtests passed em 9,07 s; duas advertências de depreciação Starlette/httpx/AnyIO. **Limite:** Ambiente preexistente; quatro casos que exigem extras de contratos ficaram pulados. Casos isolados usam mocks/fixtures locais; não são aprovação de PostgreSQL, Redis ou proxy real.

• **Procedimento:** Leitura cruzada auth/admit/queries/cache-key/cursor/CLI e schema inicial **Resultado:** As rotas chamam authorize antes de cache/ERP e quotas; as consultas comerciais incluem tenant_id; emissão administrativa verifica que todas as lojas escolhidas pertencem ao tenant. Cursor é HMAC e vincula organização, loja, período e dataset. **Limite:** Não houve prova de concorrência ou SQL com banco real nesta rodada; nenhum bypass novo foi demonstrado.

• **Procedimento:** Leitura cruzada ERP, probe, diagnostics e contracts_adapter **Resultado:** ERP usa host de configuração, SKU limitado a SKU-[0-9]{3}, store inteiro, follow_redirects=False e trust_env=False. Ferramenta de contratos recusa URL externa arbitrária/referência remota e vincula base à tentativa. Destinos de investigação aceitam somente origens loopback configuradas. **Limite:** Valores de ambiente/configuração continuam dentro da fronteira administrativa local. Não houve ensaio contra serviços externos nem exploração de rede.

• **Procedimento:** docker info --format '{{.ServerVersion}}' **Resultado:** Falhou: pipe dockerDesktopLinuxEngine ausente; daemon Linux indisponível. **Limite:** Sem build, scan, migração real, carga, rede entre containers ou integração nesta rodada.

Arquivos e trechos adicionais examinados:

• `src/api_sentinel/auth.py`

• `src/api_sentinel/cursor.py`

• `src/api_sentinel/config.py`

• `src/api_sentinel/telemetry.py (validação de entrada, safe_exception_context e caminhos de erro; leitura dirigida)`

• `src/api_sentinel/erp.py`

• `src/api_sentinel/app.py`

• `src/api_sentinel/queries.py`

• `src/api_sentinel/cli.py`

• `src/api_sentinel/db.py`

• `src/api_sentinel/rate_limit.py`

• `migrations/versions/0001_initial.py`

• `migrations/env.py`

• `alert_receiver/app.py`

• `alert_receiver/config.py`

• `alert_receiver/contracts.py`

• `alert_receiver/probe.py`

• `alert_receiver/diagnostics.py`

• `scripts/contracts_adapter.py`

• `scripts/verify_contracts.py (índice e chamadas relevantes via busca)`

• `scripts/provision_contracts.py (índice e referências a credenciais via busca)`

• `scripts/bootstrap.py`

• `scripts/review_scans.py`

• `scripts/review.py (índice, comandos e varredura de artefatos via busca)`

• `tests/unit/test_auth_admission.py (linhas 1-70)`

- `tests/contracts/test_verifier.py (linhas 1-70 e condições de skip via busca)`
- `docs/security.md`
- `compose.yml`
- `pyproject.toml`
- `Dockerfile`
- `deploy/contracts/Dockerfile`
- `deploy/proxy/nginx.conf`
- `deploy/redis/users.acl`
- `.dockerignore`
- `.gitleaks.toml`
- `.github/workflows/ci.yml`
- `.github/workflows/secrets.yml`

Limitações restantes:

- Sem revisão exaustiva de todo o histórico Git ou varredura nova de segredos. Gitleaks/Trivy e os avisos antigos citados na documentação não foram reaprovados como atuais.
- Lockfile uv.lock não recebeu auditoria linha a linha nem consulta completa de advisories; não há conclusão de ausência de CVEs.
- Não foi reexecutada migração/seed contra PostgreSQL; a leitura de permissões e isolamento não substitui prova no banco.
- Scripts grandes review.py/verify_contracts.py/provision_contracts.py tiveram inspeção dirigida, não integral.
- Não há mecanismo de backup/recovery de produção prometido pelo API Sentinel: laboratório local de dados sintéticos. A ausência de uma solução de produção não foi transformada em defeito.
- Repositório original não editado. Testes em cópia temporária com bytecode/cache desativados.

Referências adicionais:

- Fontes versionadas do repositório

containerops

Versão examinada: `57dc89474b8aa2067a1c8a925f33def0ceb27abe`.

O que foi confirmado ou encontrado implementado:

- Autorização usa proprietário derivado do Bearer, comparação constante de token e filtro owner/id no GET; chave idempotente é única por proprietário. Resposta de job não devolve payload e a sanitização de erro tem teste com marcador privado.
- A admissão e o despacho usam lock transacional compartilhado; conclusão/renovação exigem lease_token e lease vigente. Pausa impede admissão nova e retenção, permitindo drenar o trabalho existente antes do backup.
- Papéis PostgreSQL separam DML, migração e leitura de backup; restore ocorre em recursos descartáveis, usa pg_restore em transação e compara snapshot/novo job/limpeza antes de registrar aprovação.
- Pacote valida tipos/chaves/datas/origem, hashes e tamanhos; recusa links/reparse/hardlinks, nomes extras, JSON duplicado e mutação durante cópia. O archive é inspecionado sem extração e tem limites para arquivo/camadas expandidas.

• Archive transportado recusa tags compartilhadas; carregamento exige confiança e sessão, confere identidade de config/manifests/camadas e colisão de tag. OPS-05 não invalida esses controles de conteúdo ou isolamento; afeta classificação após falha.

• Scan é feito por archive sem socket Docker, verifica identidade e idade/hash da base; reports começam incomplete e HIGH/CRITICAL bloqueiam o comando. CI usa permissão contents:read, Actions por SHA e upload de caminhos selecionados, sem publicar o runtime inteiro.

• Não houve outro defeito novo confirmado nos tópicos examinados. Não se transformaram ausência de serviço externo, limitação local documentada ou falta de prova de produção em finding.

Verificações desta rodada:

• **Procedimento:** git status --short; git rev-parse HEAD; git ls-remote origin HEAD **Resultado:** Status vazio; HEAD local e remoto iguais a 57dc89474b8aa2067a1c8a925f33def0ceb27abe. **Limite:** Sem execução atual dos workflows remotos.

• **Procedimento:** Na cópia git archive, com PYTHONDONTWRITEBYTECODE=1: python -X utf8 -m unittest discover -s tests -p test_backup_package.py -q **Resultado:** 24 testes aprovados em 5,681 s. **Limite:** Fixtures/mocks; não executam pg_restore ou Docker real.

• **Procedimento:** Na cópia da aplicação, PYTHONPATH=src e Python de containerops/.venv: python -X utf8 -m pytest tests/test_security_review.py tests/test_http_contract.py tests/test_body_limit.py tests/test_api.py tests/test_domain.py -q -p no:cacheprovider **Resultado:** 98 testes aprovados em 5,37 s; duas advertências de depreciação. Uma tentativa anterior apontou por engano para app/.venv inexistente e não iniciou testes; caminho corrigido para .venv da raiz. **Limite:** Ambiente previamente instalado; banco isolado por mocks. Não houve teste contra banco real.

• **Procedimento:** python -B com unittest.TestLoader().discover para test_backup_catalog.py, test_oci_archive.py, test_runtime_context.py e test_operations.py; PYTHONDONTWRITEBYTECODE=1 **Resultado:** 53 testes executados em 9,431 s: 50 aprovados e 3 skips. A primeira execução com -X utf8 apenas no processo pai teve dois erros de decoding do stderr dos subprocessos Windows; execução normal, coerente com os subprocessos, passou. **Limite:** Não classificado como bug do produto: o erro inicial foi induzido pela configuração do comando da auditoria. Três casos condicionados a plataforma ficaram pulados. Testes de operação usam fixtures/mocks.

• **Procedimento:** python -X utf8 -B _auditoria_2026-10-13/probe_import_registration.py **Resultado:** Importação real de pacote sintético seguida de erro simulado em register**ackup d**xou diretório publicado: validation=valid, imported=false, protected=false. restore_test(trusted_source=False) atingiu stub do preflight. Com segunda cópia da mesma origem, plano keep_last=0 incluiu a primeira entre candidatas. Controle após registro correto recusou restore antes do preflight com mensagem --trusted-source. Recibo: containerops-import-registration-repro.json. **Limite:** Zero comandos externos; dump contém apenas marcador sintético. Preflight foi interrompido antes de Docker. Não executou SQL, restore, imagens nem exclusão. Proteção DACL foi substituída por fixture, pois a propriedade investigada é a ordem entre publicação e registro.

• **Procedimento:** Leitura cruzada pause/drain/cleanup/snapshot/backup/restore/release e catálogo **Resultado:** Pausa também suspende retenção de jobs; snapshot usa REPEATABLE READ; restore separa fases e invalida aprovação se cleanup falha. Encontrada lacuna específica entre publicação da importação e registro dos flags (OPS-05). **Limite:** Sem simulação de queda real de energia ou todos os pontos de falha; reprodução injeta falha de escrita do catálogo.

• **Procedimento:** docker info --format '{{.ServerVersion}}' **Resultado:** Daemon Docker Linux indisponível: pipe dockerDesktopLinuxEngine ausente. **Limite:** Sem scan atual, build, validação da rede, migração, pg_dump/pg_restore, release ou rollback reais.

Arquivos e trechos adicionais examinados:

• `app/src/containerops/config.py`

• `app/src/containerops/api.py`

- app/src/containerops/log.py
- app/src/containerops/manage.py
- app/src/containerops/database.py
- app/src/containerops/repository.py
- app/src/containerops/worker.py
- app/src/containerops/migrations/001.sql
- app/src/containerops/migrations/002.sql
- scripts/runtime_context.py
- scripts/backup_package.py (linhas 1-383, 477-698 e 732-1049; validate_manifest fora da leitura integral)
- scripts/backup_catalog.py (linhas 1-208, 333-522 e 603-694; demais índice e chamadas)
- scripts/ops.py (linhas 1-90, 550-1118, 1395-1436; comandos/índice via busca)
- scripts/supply.py (linhas 388-620 e índice/chamadas via busca)
- scripts/scan_services.py
- scripts/oci_audit.py (índice/funções via busca; comportamento de inspect_docker_archive exercitado pelos testes)
- scripts/proof.py (chamadas de release/scan e escopo via busca)
- tests/test_backup_package.py (linhas 1-175 e índice de casos)
- tests/test_backup_catalog.py (linhas 1-100 e casos de registro/interrupção via busca)
- tests/test_operations.py (linhas 1-70 e índice de casos)
- tests/test_runtime_context.py (índice de casos)
- tests/test_oci_archive.py
- app/tests/test_security_review.py
- app/pyproject.toml
- app/requirements-runtime.lock
- docker/app.Dockerfile
- docker/db/tool.sh
- docker/db/init-roles.sh
- docker/db/entrypoint.sh
- docker/db/00-platform.sh
- docker/proxy/nginx.conf
- docker/proxy/http.conf
- docker/proxy/tls.conf
- docker/proxy/locations.conf
- compose.yaml

- `.dockerignore`
- `.github/workflows/verify.yml`
- `.github/workflows/secrets.yml`
- `docs/security.md`
- `docs/supply-chain.md`
- `docs/runbooks.md (trechos de backup/importação/release/retention e comandos via busca)`
- `docs/demo.md (trechos de scope/gates via busca)`
- `README.md (trechos de limites, requisitos e provas via busca)`

Limitações restantes:

- Achado OPS-05 demonstra bypass da checagem até o preflight e elegibilidade em plano; não houve restore SQL, operação Docker ou exclusão real.
- Sem revisão completa de oci_audit.py, proof.py, supply.py ou cada ramo de retenção. O inventário indica trechos lidos; testes complementam sem substituir leitura integral.
- Não houve verificação de advisories de todas as versões nem scanner novo. Resultados históricos do repositório não foram declarados atuais.
- Pacotes/arquivos temporários usados pela prova foram removidos pelo TemporaryDirectory; somente script e recibo foram mantidos fora dos repos.
- Ambientes Python existentes foram reutilizados e caches/bytecode desativados. Repositórios originais não editados.

Referências adicionais:

- PostgreSQL 17 - pg_restore, Warning
- Fontes versionadas do repositório

cycleops

Versão examinada: `330e4c6bfd61e264956add9785557857456f5aff`.

O que foi confirmado ou encontrado implementado:

- A coleta exige HTTPS no host/caminho permitidos, rejeita redirects, desliga proxy do ambiente, limita corpo decodificado e aplica deadline total em asyncio.timeout (collection/client.py).
- O edge substitui X-Forwarded-For e a API usa limites separados de REST/SSE; requests têm identificador validado e resposta de erro sem traceback (Caddyfile e api/app.py). Isso foi conferido por código, não por ataque de rede.
- O CI declara migrações do zero, integração PostgreSQL, checks de contratos e testes Compose. Pages usa o head_sha do CI aprovado, com permissões de publicação restritas ao job deploy. A consulta pública indicou sucesso no HEAD.

Verificações desta rodada:

- **Procedimento:** git status --short; rev-parse HEAD; ls-remote origin refs/heads/main **Resultado:** Clone limpo e mesmo commit remoto da primeira rodada. **Limite:** Consulta do estado atual; não certifica commits futuros.
- **Procedimento:** round2/root_probes.py: Settings.from_env e maintain reais, com medição de banco substituída por 2048 MiB **Resultado:** 1024 suspende; NaN, inf e 1e999 são aceitos e não suspendem. **Limite:** Sem PostgreSQL; prova da validação/comparação, não de esgotamento real de disco.

• **Procedimento:** round2/root_probes.py: ArchiveWriter real; prune com limite reduzido para 4096 bytes **Resultado:** Arquivo ativo chegou a 19402 bytes após uma escrita, sem rejeição ou nova limpeza. **Limite:** Dado sintético em diretório temporário. Não demonstra que o feed Oslo atinja 500 MiB nem propõe carga contra ele.

• **Procedimento:** pytest tests/test_worker_main.py tests/test_worker_archive.py -q -p no:cacheprovider -o addopts="" **Resultado:** 22 testes passaram em 4,37 s no Python 3.14 existente. **Limite:** Somente esses dois módulos; não reexecuta integração/Compose.

• **Procedimento:** API pública GitHub: actions/runs?per_page=5 **Resultado:** CI 38476397466 e Pages 38477189965 concluídos com success no HEAD auditado. **Limite:** Estado reportado pelo GitHub; logs completos e cada artefato do runner não foram reavaliados.

Arquivos e trechos adicionais examinados:

• `AGENTS.md (instrução local, não publicação)`

• `SECURITY.md`

• `apps/backend/src/cycleops/worker/main.py (configuração e liderança)`

• `apps/backend/src/cycleops/worker/processor.py`

• `apps/backend/src/cycleops/worker/archive.py`

• `apps/backend/src/cycleops/collection/client.py`

• `apps/backend/src/cycleops/adapters/postgres/maintenance.py`

• `apps/backend/src/cycleops/api/app.py`

• `apps/backend/src/cycleops/jsonlog.py`

• `infra/caddy/Caddyfile`

• `.github/workflows/ci.yml`

• `.github/workflows/pages.yml`

• `apps/web/package.json`

• `apps/backend/tests/test_worker_main.py`

• `apps/backend/tests/test_worker_archive.py`

Limitações restantes:

• Docker continua indisponível; não houve novo ensaio de recuperação do worker, disco cheio, SSE sob carga ou restore PostgreSQL.

• O teste de arquivo usa limiar reduzido e é compatível com a exceção intencional já coberta pelo projeto. Não é falha de segurança remota.

• Não foi repetida varredura de vulnerabilidades de todas as dependências nem de imagens; CI verde não é certificação de segurança.

Referências adicionais:

• Python: math.isfinite

• CycleOps CI no commit auditado

• CycleOps publicação Pages no commit auditado

quake-exposure

Versão examinada: `a928c1f6e7faa5ee542f127e243320515c9ceab4`.

O que foi confirmado ou encontrado implementado:

• O runbook distingue backup de duas bases, reaplicação do ledger, restauração da origem e resíduos em Kafka/WAL/dumps. A política manual de expiração está explicitada; não foi convertida em defeito por ausência de automação.

• restore_verify recusa banco existente sem a marca esperada, verifica SHA dos dumps e nega conexão da API ao banco restaurado.

• O cliente USGS verifica TLS, rejeita redirecionamento e limita hosts/tamanho; get_product confere length e SHA declarados ao baixar. QEX-04 fica na reutilização posterior do arquivo, não na verificação do download.

• A ausência de login não é falha de autorização no escopo declarado: Compose publica a API em loopback e o módulo explica que tenant funciona como filtro. Exposição pública de dados reais exigiria outro modelo, que não foi presumido nesta revisão.

Verificações desta rodada:

• **Procedimento:** round2/root_probes.py: BlobStore real em pasta temporária **Resultado:** put(A), substituição por gzip válido B, get(A) retorna B; novo put(A) devolve o mesmo hash/caminho sem reparar nem recusar B. **Limite:** Prova do armazenamento local; não é exploração remota, falha real do volume publicado ou execução de cálculo/PostGIS.

• **Procedimento:** git status --porcelain; rev-parse HEAD; API pública de Actions **Resultado:** Cópia de auditoria limpa; CI 38202998983 com success no HEAD. A última publicação Demo REPLAY listada está no commit 0979d3118a2d3fc172dcdcef90b310db9bec0fb5. **Limite:** Deploy tem filtro de paths web/**; estar em commit anterior após mudanças apenas de backend não caracteriza erro. Não foram baixados todos os logs.

• **Procedimento:** Leitura cruzada de backup, erasure, ledger, API, Compose e runbook **Resultado:** Ledger é sincronizado antes do bloqueio local; restore valida hashes, exige ledger presente e usa banco marcado/sem CONNECT da API. API é explicitamente local, sem login, e tenant é filtro. **Limite:** Inspeção estática. Não atesta adequação jurídica, anonimização completa nem desastre/recovery executado.

Arquivos e trechos adicionais examinados:

• `src/quakeexposure/privacy/ledger.py`

• `src/quakeexposure/privacy/backup.py`

• `src/quakeexposure/privacy/erasure.py`

• `src/quakeexposure/ingestion/blobs.py`

• `src/quakeexposure/ingestion/http.py`

• `src/quakeexposure/ingestion/projection.py (project_pending/project_capture)`

• `src/quakeexposure/exposure/worker.py (claim,run,run_pending,_load_grid)`

• `src/quakeexposure/db.py`

• `src/quakeexposure/api/__init__.py (contrato local e dependência db)`

• `compose.yaml (serviços de dados, worker e API)`

• `Dockerfile`

• `docs/operacao.md (recuperação CDC, exclusão e backups)`

• `README.md (operação e limites)`

• `.github/workflows/ci.yaml`

• `.github/workflows/pages.yaml`

Limitações restantes:

- Sem teste de restauração, privacidade concorrente, CDC real, PostGIS ou volume Docker nesta rodada.
- Leitura de políticas e jobs não equivale a certificação de segurança ou conformidade de proteção de dados.
- Nenhum novo achado é atribuído a IA. Bibliotecas, tags de imagens e cabeçalhos foram examinados como configuração, sem alegação de CVE não verificada.

Referências adicionais:

- QuakeExposure CI no commit auditado

delta

Versão examinada: `cffa94f9e2e9a4d672e994021755f3f8ec125fc7`.

O que foi confirmado ou encontrado implementado:

- Contribuição atribuída apenas ao diff do autor; o tamanho, a arquitetura e os controles upstream não foram usados como evidência de sua autoria.
- Estar aberto não é defeito do PR nem comprovação de aceitação. Não surgiu novo defeito autoral nos materiais revisitados.

Verificações desta rodada:

- **Procedimento:** git show --stat do commit autoral; consulta atual do PR na API pública **Resultado:** Diff autoral permanece limitado ao mesmo exemplo; PR segue aberto e merged_at nulo, com mesmo SHA da primeira rodada. **Limite:** Sem reexecução da suíte completa upstream; as provas funcionais anteriores permanecem registradas.

Arquivos e trechos adicionais examinados:

- `python/delta/tables.py nas versões padrão e autoral`
- `diff completo do commit autoral`
- `refs remotos e metadados públicos do PR`

Limitações restantes:

- A revisão complementar dos forks continua restrita à contribuição autoral; não é auditoria da base upstream.

fastapi-pagination

Versão examinada: `b626a76fd4f32d4f9328d3bfc1376bfa16a68109`.

O que foi confirmado ou encontrado implementado:

- Contribuição atribuída apenas ao diff do autor; o tamanho, a arquitetura e os controles upstream não foram usados como evidência de sua autoria.
- Estar aberto não é defeito do PR nem comprovação de aceitação. Não surgiu novo defeito autoral nos materiais revisitados.

Verificações desta rodada:

- **Procedimento:** git show --stat do commit autoral; consulta atual do PR na API pública **Resultado:** Diff autoral permanece limitado ao mesmo exemplo; PR segue aberto e merged_at nulo, com mesmo SHA da primeira rodada. **Limite:** Sem reexecução da suíte completa upstream; as provas funcionais anteriores permanecem registradas.

Arquivos e trechos adicionais examinados:

- `docs/learn/tutorial_advanced/custom_paginate.md nas versões padrão e autoral`
- `diff completo do commit autoral`

- `refs remotos e metadados públicos do PR`

Limitações restantes:

- A revisão complementar dos forks continua restrita à contribuição autoral; não é auditoria da base upstream.

Plano complementar de correção

Recorte histórico dos nove achados adicionais. A tabela única com os 47 itens está na Parte 5 do relatório principal; os resultados posteriores estão na revisão.

Os três P1 desta rodada tratam de barreiras operacionais diferentes. Podem ser feitos em paralelo por repositório. Integre cada mudança ao teste que reproduz seu modo de falha; em seguida atualize as instruções e evidências correspondentes.

ID	Repositório	Problema	Categoria	Prioridade	Esforço	Arquivo
OPS-05	containerops	Importação interrompida após publicar o pacote perde as marcas de origem externa e proteção	Segurança, backup e recuperação de falhas	P1	médio	scripts/ops.py:1422-1431; scripts/backup_package.py:661-687; scripts/backup_catalog.py:350-360,415-425; scripts/ops.py:760-766
QEX-04	quake-exposure	Blob armazenado é reutilizado sem conferir a identidade SHA-256	Integridade de armazenamento e recuperação	P1	médio	src/quakeexposure/ingestion/blobs.py:13-31; ingestion/projection.py:117; exposure/worker.py:184-189
LOJ-07	loja-assistente	Login permite concorrência de Argon2 incompatível com o limite de memória do contêiner	Autenticação e disponibilidade	P1	médio	backend/src/loja_assistente/auth/service.py:18; backend/src/loja_assistente/auth/routes.py:68-70; backend/src/loja_assistente/database.py:13-24; backend/src/loja_assistente/auth/admission.py:16-18; compose.yaml:46
LOJ-08	loja-assistente	Erro de configuração pode incluir partes de valores sensíveis no diagnóstico de inicialização	Privacidade e configuração	P2	baixo	backend/src/loja_assistente/config.py:11,13,17,21,55-67
EVD-07	evidencedesk	Erros rejeitados pelo limite de corpo apresentam dois IDs de rastreamento	Observabilidade e contrato de erro	P2	baixo	backend/src/evidencedesk/body_limit.py:28-43; backend/src/evidencedesk/app.py:73,107-112
REC-08	gestao-recebiveis	Campos textuais JSON aceitam NUL que a persistência não representa	Validação de entrada e confiabilidade da API	P2	médio	backend/src/gestao_recebiveis/schemas.py:19; backend/src/gestao_recebiveis/auth.py:64; backend/src/gestao_recebiveis/app.py:79
REC-07	gestao-recebiveis	O roteiro de atualização descreve a migração atual como apenas aditiva	Documentação operacional e migrações	P2	baixo	docs/security.md:32; backend/migrations/versions/e71c2f_partial_payments.py:14
CYC-04	cycleops	Configuração aceita quota não finita e desativa a suspensão por tamanho	Configuração e limites de recursos	P2	baixo	apps/backend/src/cycleops/worker/main.py:95-99; worker/processor.py:119-135
CYC-05	cycleops	Documentação apresenta a limpeza do arquivo bruto como teto rígido de armazenamento	Documentação operacional e retenção	P2	baixo	SECURITY.md:11; apps/backend/src/cycleops/worker/archive.py:1-2,51-60,67-79,98

Checklist complementar e critério de encerramento

evidencedesk

• [ ] **EVD-07 (P2)** - Definir um único request_id por requisição e reutilizá-lo no limitador, no envelope JSON, no cabeçalho e no evento de conclusão. Começar por `backend/src/evidencedesk/body_limit.py`.

• [ ] Validar **EVD-07** - Comparar o mesmo request_id no cabeçalho, corpo do 413 e evento de conclusão, com e sem Content-Length.

gestao-recebiveis

• [ ] **REC-08 (P2)** - Rejeitar NUL nos campos que serão persistidos/consultados como texto ou JSONB, antes de executar o domínio. Começar por `backend/src/gestao_recebiveis/schemas.py`.

• [ ] Validar **REC-08** - Enviar NUL em e-mail, observação, chave, motivo, busca e nome do upload; exigir 422 antes do domínio, sem efeito persistido.

• [ ] **REC-07 (P2)** - Atualizar o procedimento para identificar a revisão financeira, suas condições de recusa e os limites de retorno ao schema anterior. Começar por `docs/security.md`.

• [ ] Validar **REC-07** - Conferir e71c2f no roteiro: locks, validação de legado e recusa do downgrade; preservar os dados no ensaio de atualização.

loja-assistente

• [ ] **LOJ-07 (P1)** - Adicionar limite explícito de verificações de senha simultâneas, dimensionado com a memória do contêiner e o número de processos, mantendo a admissão durável já existente. Começar por `backend/src/loja_assistente/auth/service.py`.

• [ ] Validar **LOJ-07** - Saturar o limite de duas verificações de senha, exigir recusa antes do Argon2 e liberação da vaga depois de sucesso ou exceção.

• [ ] **LOJ-08 (P2)** - Ocultar valores de entrada nas mensagens de validação de Settings e modelar campos sensíveis com tipos que mascaram sua representação. Começar por `backend/src/loja_assistente/config.py`.

• [ ] Validar **LOJ-08** - Provocar erro de configuração com marcador sintético e exigir que o valor não apareça no diagnóstico nem na representação de Settings.

containerops

• [ ] **OPS-05 (P1)** - Fazer a classificação de origem e a proteção sobreviverem à interrupção entre a publicação do diretório e o catálogo. Enquanto um pacote importado não tiver registro concluído, tratá-lo conservadoramente como pendente/protegido e nunca presumir origem local. Recuperar o registro antes de liberar retenção e manter o reconhecimento explícito de confiança para restore. Começar por `scripts/ops.py`.

• [ ] Validar **OPS-05** - Interromper após publicar o pacote e antes do catálogo; retenção deve preservar o pacote externo pendente, com recuperação de origem/proteção antes do restore.

cycleops

• [ ] **CYC-04 (P2)** - Rejeitar números não finitos ao ler CYCLEOPS_DB_QUOTA_MB, mantendo o erro de configuração e o código de saída já existentes. Começar por `apps/backend/src/cycleops/worker/main.py`.

• [ ] Validar **CYC-04** - Rejeitar NaN, infinidades, overflow, zero e negativos; aceitar quota finita positiva como 0.5.

• [ ] **CYC-05 (P2)** - Corrigir SECURITY.md e a docstring para explicar a política real. Manter a preservação do arquivo ativo; um teto físico exigiria decisão separada de rotação por tamanho e comportamento quando não houver espaço. Começar por `SECURITY.md`.

• [ ] Validar **CYC-05** - Comprovar que limpeza remove somente arquivos antigos e pode exceder o limiar quando resta o arquivo ativo; documentação não deve prometer teto físico.

quake-exposure

• [ ] **QEX-04 (P1)** - Verificar SHA-256 dos bytes descomprimidos contra control.capture antes de projetar/calcular. Validar também o arquivo existente no caminho rápido de put e recusar divergência com erro operacional explícito. Começar por `src/quakeexposure/ingestion/blobs.py`.

• [ ] Validar **QEX-04** - Usar gzip válido com bytes divergentes do SHA-256 e arquivo existente adulterado; exigir falha antes da projeção e preservar o suspeito.

• [ ] Em VDP-02, preservar acesso do report-server ao arquivo publicado e validar o GET no UID do container.

• [ ] Com Docker disponível, executar as integrações pendentes relevantes às correções em ambientes descartáveis e anexar resultados com commit e comando.

• [ ] Atualizar textos substitutos somente depois de o comportamento correspondente existir; não converter um plano em afirmação de teste aprovado.

O perfil e os dois forks não ganharam novos itens de implementação nesta rodada. As propostas anteriores e o acompanhamento do status dos PRs continuam válidos. A revisão complementar não apresentou indício suficiente para acusar geração por IA, vazamento real de credencial, execução de pacote externo ou corrupção financeira.

Registro da segunda rodada

Achados estruturados: `_auditoria_2026-10-13/supplement_root.json`, `supplement_apps.json`, `supplement_backend_ops.json` e `supplement_data_billing.json`. O inventário atualizado está em `inventario_github_segunda_rodada.json`. Os scripts, recibos e consultas do responsável principal estão em `round2/`; os agentes registraram seus recibos com caminhos discriminados nos procedimentos acima. O complemento registra controles confirmados, defeitos novos e limites de observação separadamente.